FOM Hochschule für Oekonomie & Management

Hochschulzentrum Köln

Seminararbeit

im Studiengang KI & Business Analytics

zur Erlangung des Grades eines

Master of Science (M. Sc.)

über das Thema

Versionierung von Datenmodellen

von

Florian Stevener

Betreuer	Prof. Dr.-Ing. Peter Steininger
Matrikelnummer	839237
Abgabedatum	24.05.2026

Inhaltsverzeichnis

Abbildungsverzeichnis III

Quelltexte IV

1 Einleitung 1

2 Theoretische Grundlagen 2

2.1 Datenbankschema 2

2.2 Schemamodifikation 5

2.3 Schema-Evolution 6

2.4 Schema-Versionierung 7

3 Methoden der Schema-Versionierung 9

3.1 Dimensionen der Schema-Versionierung 9

3.1.1 Partielle und vollständige Schema-Versionierung 9

3.1.2 Temporale Schema-Versionierung 12

3.1.3 Speicherkonzept 17

3.1.4 Interaktion zwischen Instanz und Schema 21

3.2 DBMS Support und Tooling 26

3.3 Ausblick 27

Literaturverzeichnis 28

Abbildungsverzeichnis

1 ANSI-SPARC-Architektur 4
2 Zustand vor der Schemamodifikation 5
3 Zustand nach der Schemamodifikation 6
4 Zustand nach der Schema-Evolution 7
5 Zustand nach der Schema-Versionierung 9
6 Partielle Schema-Versionierung 10
7 Vollständige Schema-Versionierung 11
8 Transaktionszeitbasierte Schema-Versionierung 15
9 Gültigkeitszeitbasierte Schema-Versionierung 16
10 Bitemporale Schema-Versionierung 17
11 Multi-Pool-Speicherung 18
12 Single-Pool-Speicherung 20
13 Gehaltshistorie des Mitarbeiter *Brown* 23
14 Synchrones Management 24
15 Asynchrones Management 25
16 Testpyramide 27

Quelltexte

1 DDL-Statements 6

1 Einleitung

Änderungen an Datenmodellen treten in datengetriebenen Informationssystemen regelmäßig auf und sind als natürlicher Bestandteil ihrer Weiterentwicklung zu verstehen. Ursachen hierfür liegen unter anderem in sich verändernden Nutzeranforderungen, der Anpassung an neue Geschäftslogiken oder in externen Rahmenbedingungen wie gesetzlichen Vorgaben (nach BRAHMIA, GRANDI, OLIBONI 2024a, S. 2430001-1). Diese strukturellen Anpassungen stellen besondere Anforderungen an die Verwaltung von Datenbeständen, da bestehende Daten und Anwendungen weiterhin funktionsfähig bleiben müssen (nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-2).

Ein zentraler Ansatz zur Bewältigung dieser Problematik ist die Schema-Versionierung. Dieses Konzept sieht vor, verschiedene Versionen eines Datenbankschemas parallel zu verwalten und die zugehörigen Daten konsistent diesen Versionen zuzuordnen. Dadurch können sowohl historische Datenbestände als auch ältere Anwendungen weiterhin genutzt werden, ohne dass Informationen verloren gehen oder unmittelbare Migrationen erforderlich sind. Schema-Versionierung wurde sowohl für klassische relationale Datenbanksysteme als auch für neuere Datenbankparadigmen umfassend untersucht. Obwohl eine Vielzahl wissenschaftlicher Arbeiten zu diesem Thema existiert, ist die praktische Unterstützung in Datenbanksystemen bislang nur eingeschränkt ausgeprägt (nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-1 u. S. 2430002-3).

Vor diesem Hintergrund bietet die vorliegende Seminararbeit eine Einführung in die grundlegenden Konzepte der Schema-Versionierung, wie sie in der wissenschaftlichen Literatur behandelt werden. Der Fokus der Arbeit liegt ausschließlich auf der Versionierung von Datenbankschemata in relationalen Datenbanken.

Methodisch basiert die Arbeit auf einer Literaturanalyse einschlägiger Fachquellen, eigene empirische Erhebungen werden nicht durchgeführt.

Die Arbeit ist wie folgt aufgebaut: Kapitel 2 definiert zunächst die zentralen Begriffe *Datenbankschema*, *Schemamodifikation*, *Schema-Evolution* sowie *Schema-Versionierung*

und grenzt diese voneinander ab. Kapitel 3 behandelt anschließend grundlegende Konzepte der Schema-Versionierung, wie sie in der wissenschaftlichen Literatur beschrieben werden und stellt ausgewählte Softwarelösungen vor. Abschließend erfolgt ein kurzer Ausblick.

2 Theoretische Grundlagen

In diesem Kapitel wird für den Begriff *Schema-Versionierung* eine für diese Arbeit gültige Begriffsbestimmung auf Grundlage etablierter wissenschaftlicher Definitionen vorgenommen. Darüber hinaus werden verwandte Konzepte erläutert, die der Schema-Versionierung inhaltlich nahe stehen und daher häufig mit ihr verwechselt oder vermischt werden.

2.1 Datenbankschema

Das *Datenbankschema* definiert die Struktur der Datenbank (nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-2). Es ist eine strukturierte Menge von Metadaten, wird mit der SQL Data Definition Language (DDL) erzeugt und im Systemkatalog des *Datenbankmanagementsystems* (kurz *DBMS*) gespeichert (vgl. ELMASRI, NAVATHE 2011, S. 33; BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-2). Im Folgenden wird der Begriff *Schema* synonym für das Datenbankschema verwendet. Schemata sind für die Abfrage, Aktualisierung, Integration und Konvertierung von Daten erforderlich (nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-2). Die ANSI-SPARC-Architektur definiert Schemata auf drei verschiedenen Abstraktionsebenen: das *interne Schema*, das *konzeptionelle Schema* und das *externe Schema* (vgl. Abb. 1). Die folgenden Ausführungen zu den Schemaebenen der ANSI-SPARC-Architektur basieren auf ELMASRI und NAVATHE (2011, S. 33 ff.).

Das *interne Schema* definiert die physische Struktur der Datenbank. Es enthält vollständige Informationen über die physische Speicherung der Daten auf dem Datenträger.

Das *konzeptionelle Schema* definiert die logische Struktur der Datenbank. Es abstrahiert die Details der physischen Speicherung und bezieht sich ausschließlich auf das konzep-

tionelle Design der Datenbank. Es beschreibt Relationen, Attribute, Datentypen, Beziehungen zwischen Entitäten und Integritätsbedingungen. Das konzeptionelle Schema legt damit fest, wie die Daten in der Datenbank angeordnet sind und in welcher Beziehung sie zueinander stehen.

Das *externe Schema* definiert die benutzerspezifische Sicht auf die Datenbank. Es legt fest, auf welche Teilmenge des konzeptionellen Schemas ein Benutzer zugreifen darf. Für verschiedene Benutzergruppen können unterschiedliche externe Schemata existieren.

Die Abstraktionsebenen der ANSI-SPARC-Architektur garantieren Datenunabhängigkeit. Schemaänderungen auf einer Ebene des Datenbanksystems haben damit keine Auswirkungen auf das Schema der nächsthöheren Ebene. Es wird zwischen *physischer Datenunabhängigkeit* und *logischer Datenunabhängigkeit* differenziert. Physische Datenunabhängigkeit beschreibt die Fähigkeit, Änderungen am internen Schema eines Datenbanksystems vorzunehmen, ohne das konzeptionelle Schema anpassen zu müssen. Logische Datenunabhängigkeit liegt vor, wenn das konzeptionelle Schema geändert werden kann, ohne das externe Schema anpassen zu müssen (nach ELMASRI, NAVATHE 2011, S. 35 f.). Die in dieser Arbeit vorgestellten Konzepte beziehen sich primär auf das konzeptionelle Schema, wenngleich Schema-Versionierung alle drei Ebenen der ANSI-SPARC-Architektur betreffen kann (vgl. SILBERSCHATZ, KORTH, SUDARSHAN 2019, S. 12 f.; BRAHMIA, GRANDI, OLIBONI 2024b).

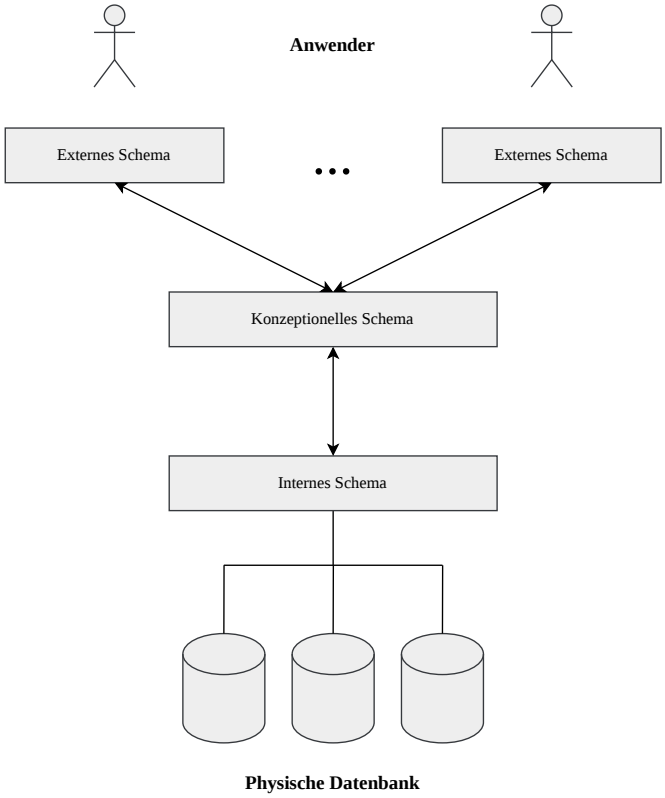


Abbildung 1: ANSI-SPARC-Architektur

Quelle: ELMASRI, NAVATHE 2011, S. 34

Das Schema ist eine intensionale Beschreibung der Datenbank und enthält selbst keine Daten. Die *Datenbankinstanz* (im Folgenden kurz *Instanz*) hingegen repräsentiert den konkreten Zustand einer Datenbank zu einem bestimmten Zeitpunkt. Sie wird auch als Extension bezeichnet und enthält die gespeicherten Daten gemäß den Strukturen und Integritätsbedingungen des Schemas. Jedes Einfügen, Löschen oder Aktualisieren von Daten führt zu einem neuen Zustand und damit zu einer neuen Instanz. Das Schema bleibt dabei konstant und ist zeitlich wesentlich stabiler als die Instanz (nach ELMASRI, NAVATHE 2011, S. 32 f.). Erst wenn die Struktur der Datenbank modifiziert werden soll, muss das Schema angepasst werden. Dies kann beispielsweise eintreten, wenn sich die Anforderungen an die Applikation ändern, die mit der Datenbank interagiert (vgl. BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-2). Die Kapitel 2.2 bis 2.4 gehen näher darauf ein, wie Änderungen an Schemata in einem DBMS unterstützt werden können.

2.2 Schemamodifikation

Änderungen an der Definition des Schemas einer bestehenden Datenbank, werden auch als *Schemamodifikation* bezeichnet (nach RODDICK 1995, S. 384). Schemamodifikation umfasst das Hinzufügen, Modifizieren und Entfernen von Elementen des Schemas (nach BRAHMIA, GRANDI, OLIBONI 2024a, S. 2430001-2). Bei der Schemamodifikation wird das bestehende Schema inklusive der zugehörigen Instanz verworfen und durch ein neues, leeres Schema ersetzt. Aspekte wie Datenerhalt und Schemakonsistenz werden dabei – im Gegensatz zur Schema-Evolution und Schema-Versionierung – nicht berücksichtigt (nach BRAHMIA, GRANDI, OLIBONI 2024a, S. 2430001-6 ff.).

Die Abbildungen 2 und 3 verdeutlichen den Ablauf der Schemamodifikation in einer relationalen Datenbank. Das folgende Beispiel sowie die Beispiele in den Kapiteln 2.3 und 2.4 wurden in Anlehnung an DE CASTRO, GRANDI und SCALAS (1997, S. 250 ff.) erstellt.

Ausgangspunkt ist die Schemaversion SV_1 mit dem entsprechenden Eintrag im Systemkatalog des DBMS. SV_1 besteht aus der Relation *EMPLOYEE* mit den Attributen *NAME*, *ADDRESS* und *CITY*. Die Instanz I_1 umfasst zwei Tupel (vgl. Abb. 2).

SV_1

EMPLOYEE (NAME, ADDRESS, CITY)

I_1

NAME	ADDRESS	CITY
Brown	King's Road 15	London
Rossi	Via Veneto 7	Rome

Abbildung 2: Zustand vor der Schemamodifikation

Quelle: DE CASTRO, GRANDI, SCALAS 1997, S. 251

Das Schema wird mit Hilfe von DDL-Statements verändert (vgl. Quelltext 1). Zunächst wird das Attribut *ADDRESS* aus der Relation entfernt. Anschließend wird ein neues Attribut *PHONE* hinzugefügt.

```sql
ALTER TABLE EMPLOYEE
    DROP COLUMN ADDRESS;

ALTER TABLE EMPLOYEE
    ADD COLUMN PHONE CHAR(8);
```

Quelltext 1: DDL-Statements

Quelle: DE CASTRO, GRANDI, SCALAS 1997, S. 251

Dadurch entsteht die neue Schemaversion SV_2 mit leerer Instanz I_2 und angepasstem Eintrag im Systemkatalog (vgl. Abb. 3). Die ursprüngliche Schema-Version SV_1 existiert nach der Modifikation nicht mehr. Auch die mit SV_1 verbundene Instanz I_1 geht verloren.

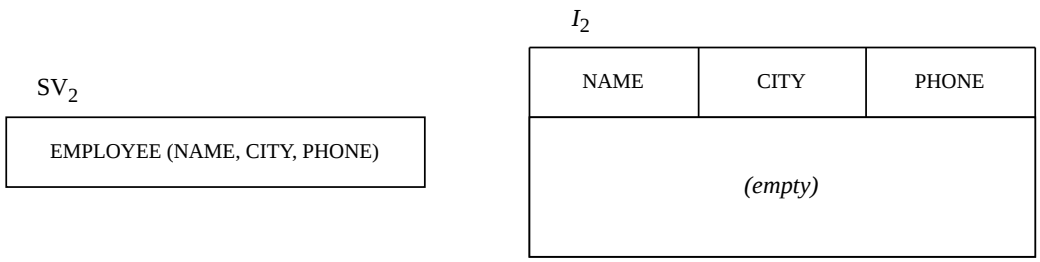


Abbildung 3: Zustand nach der Schemamodifikation

Quelle: DE CASTRO, GRANDI, SCALAS 1997, S. 251

Das Beispiel zeigt damit die charakteristischen Eigenschaften einer klassischen Schemamodifikation: Änderungen werden direkt auf dem bestehenden Schema durchgeführt, ohne frühere Schemaversionen oder historische Instanzen beizubehalten. Anwendungen und Anfragen müssen anschließend zwingend mit der neuen Struktur kompatibel sein (vgl. BRAHMIA, GRANDI, OLIBONI 2024a, S. 2430001-6 ff.).

2.3 Schema-Evolution

Schema-Evolution bezeichnet die Änderung des Schemas einer bestehenden Datenbank, wobei die bereits gespeicherten Daten durch geeignete Migrationsoperationen an das neue Schema angepasst werden. Sie grenzt sich von der Schemamodifikation dadurch ab, dass die bestehenden Daten erhalten bleiben und in das neue Schema überführt werden, anstatt

sie zu verwerfen. Dennoch kann es im Rahmen der Schema-Evolution, beispielsweise bei der Entfernung von Attributen oder der Änderung von Datentypen, zu einem teilweisen Informationsverlust kommen. Auch die Kompatibilität des neuen Schemas mit Anwendungen, die auf Basis des alten Schemas entwickelt wurden, ist durch Schema-Evolution nicht garantiert (vgl. RODDICK 1995, S. 384; DE CASTRO, GRANDI, SCALAS 1997, S.250; BRAHMIA, GRANDI, OLIBONI 2024a, S. 2430001-7 f.).

Abbildung 4 zeigt das aus den DDL-Statements in Quelltext 1 resultierende Schema bei einem DBMS, das Schema-Evolution unterstützt. Die ursprüngliche Schemaversion SV_1 wird verworfen und vollständig durch die neue Schemaversion SV_2 ersetzt. Ebenso wird die bestehende Instanz I_1 vollständig durch die neue Instanz I_2 ersetzt. Die Daten der Attribute *NAME* und *CITY* werden auf Basis von I_1 in die Instanz I_2 der neuen Schemaversion SV_2 übernommen. Die Informationen des Attributes *ADDRESS* gehen hingegen verloren, da dieses kein Bestandteil der Schemaversion SV_2 ist. Für das neu eingeführte Attribut *PHONE* liegen zunächst keine Werte vor, sodass die entsprechenden Tupel mit NULL-Werten belegt werden (vgl. BRAHMIA, GRANDI, OLIBONI 2024a, S. 2430001-7 f.).

SV_2

EMPLOYEE (NAME, CITY, PHONE)

I_2

NAME	CITY	PHONE
Brown	London	*Null*
Rossi	Rome	*Null*

Abbildung 4: Zustand nach der Schema-Evolution

Quelle: DE CASTRO, GRANDI, SCALAS 1997, S. 251

2.4 Schema-Versionierung

Schema-Versionierung bezeichnet die Fähigkeit eines Datenbanksystems, verschiedene Zustände eines Schemas über die Zeit hinweg zu verwalten und zugänglich zu machen. Dabei werden Änderungen am Schema historisch gespeichert, sodass frühere und aktuelle Versionen weiterhin genutzt werden können (nach RODDICK 1995, S. 384). Im Unterschied zur Schema-Evolution wird das bestehende Schema nicht ersetzt, sondern durch

eine zusätzliche Schemaversion ergänzt. Folglich können mehrere Schemaversionen parallel im DBMS zur Laufzeit existieren. Ein wesentliches Merkmal ist zudem die Möglichkeit, Daten sowohl über aktuelle als auch historische Schemata abzurufen. Das ermöglicht Anwendungen auf jene Schemaversion zuzugreifen, für die sie entwickelt wurden, wodurch ihre Funktionsfähigkeit erhalten bleibt. Neue Daten werden dabei in der Regel nach der aktuellen Schemaversion gespeichert. Ein zentrales Ziel der Schema-Versionierung ist es, Informationsverlust zu vermeiden und den Betrieb bestehender Anwendungen weiterhin zu gewährleisten (vgl. BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-2 f. u. S. 2430002-5 ff.).

Schema-Versionierung ist komplexer als die reine Schema-Evolution, da sämtliche Schemaversionen innerhalb eines DBMS gemeinsam mit den zugehörigen Instanzen und abhängigen Komponenten koexistieren und konsistent betrieben werden müssen (nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-14).

Abbildung 5 zeigt das aus den DDL-Statements in Quelltext 1 resultierende Schema bei einem DBMS, das Schema-Versionierung unterstützt. Im Gegensatz zur Schema-Evolution koexistiert die neue Schemaversion SV_2 mit der ursprünglichen Schemaversion SV_1 innerhalb des DBMS. Dies wird unter anderem durch die beiden Einträge im Systemkatalog deutlich. Die Instanz I_1 der Schemaversion SV_1 bleibt vollständig erhalten. Für SV_2 werden die Daten analog zur Schema-Evolution an die neue Schemastruktur angepasst, sodass die Instanz I_2 entsteht. Dadurch können Anwendungen, die auf Basis von SV_1 oder SV_2 entwickelt wurden, weiterhin ohne Anpassungen betrieben werden (vgl. BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-5 ff.).

SV_1

EMPLOYEE (NAME, ADDRESS, CITY)

I_1

NAME	ADDRESS	CITY
Brown	King's Road 15	London
Rossi	Via Veneto 7	Rome

SV_2

EMPLOYEE (NAME, CITY, PHONE)

I_2

NAME	CITY	PHONE
Brown	London	*Null*
Rossi	Rome	*Null*

Abbildung 5: Zustand nach der Schema-Versionierung

Quelle: DE CASTRO, GRANDI, SCALAS 1997, S. 252

3 Methoden der Schema-Versionierung

Im folgenden Kapitel werden die grundlegenden Konzepte der Schema-Versionierung vorgestellt, die in zahlreichen Forschungsarbeiten als Basis unterschiedlicher Lösungsansätze dienen. Ziel des Kapitels ist die Vermittlung eines allgemeinen Verständnisses der zugrunde liegenden Prinzipien und Mechanismen der Schema-Versionierung, ohne dabei auf die detaillierte Implementierung einzelner Ansätze einzugehen. Aus diesem Grund werden spezifische Forschungsergebnisse und prototypische Implementierungen im Rahmen dieser Arbeit nicht vertieft behandelt.

3.1 Dimensionen der Schema-Versionierung

3.1.1 Partielle und vollständige Schema-Versionierung

Im Hinblick auf die Lese- und Schreib-Operationen, welche auf die Daten eines Schemas ausgeführt werden können, wird in der Literatur zwischen *partieller Schema-Versionierung* und *vollständiger Schema-Versionierung* differenziert (nach RODDICK 1995, S. 384).

Die *partielle Schema-Versionierung* ermöglicht das Abrufen von Daten über alle vergangenen und zukünftigen Schemaversionen hinweg. Änderungen an den Daten können jedoch nur über eine einzige – in der Regel die aktuelle – Schemaversion vorgenommen werden. Daher gewährleistet die partielle Schema-Versionierung nicht, dass bestehende Anwendungen nach einer Schemaänderung weiterhin fehlerfrei funktionieren, sofern diese Anwendungen Daten modifizieren (vgl. RODDICK 1995, S. 384; BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-15). Abbildung 6 verdeutlicht, dass die Instanzen I_1 und I_2 sowohl über SV_1 als auch über SV_2 abrufbar sind. Schreib-Operationen können jedoch nur über SV_2 auf I_2 ausgeführt werden. Anwendungen, die Schreib-Operationen über SV_1 ausführen, funktionieren daher nach der Einführung von SV_2 nicht mehr korrekt. Ab diesem Zeitpunkt sind Schreib-Operationen nur noch über SV_2 möglich (vgl. RODDICK 1995, S. 384; BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-7).

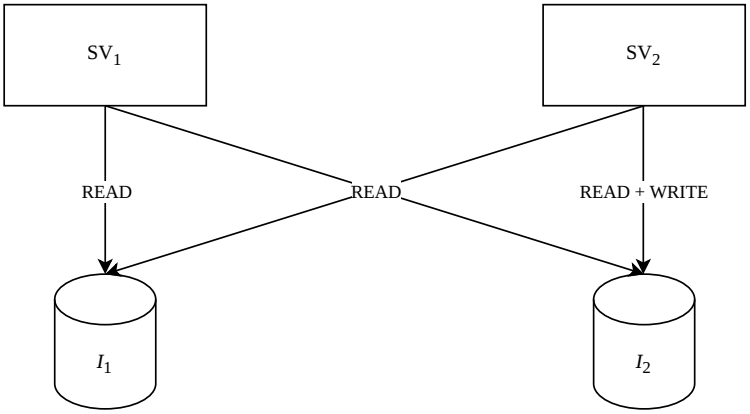


Abbildung 6: Partielle Schema-Versionierung

Quelle: Eigene Darstellung nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-7

Die *vollständige Schema-Versionierung* hingegen ermöglicht sowohl das Lesen als auch das Schreiben von Daten über alle vergangenen und zukünftigen Schemaversionen hinweg. Dadurch bleiben bereits kompilierte Anwendungen auch nach beliebigen Schemaänderungen weiterhin voll funktionsfähig (vgl. RODDICK 1995, S. 384; BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-15). Das Lesen und Schreiben ist hierbei über SV_1 und SV_2 auf die Instanzen I_1 und I_2 möglich (vgl. Abb. 7) (vgl. RODDICK 1995, S. 384; BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-7).

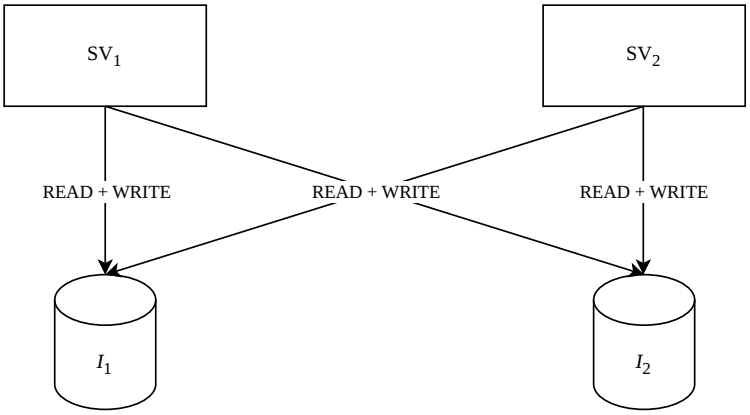


Abbildung 7: Vollständige Schema-Versionierung

Quelle: Eigene Darstellung nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-7

Die Kompatibilität zwischen Instanzen und Schemaversionen ist ein zentrales Problem der Schema-Versionierung. In diesem Kontext unterscheidet die Literatur zwischen *Rückwärtskompatibilität* und *Vorwärtskompatibilität*. Rückwärtskompatibilität bedeutet, dass Daten, die unter einer früheren Schemaversion erstellt wurden, von einer Anwendung verarbeitet werden können, die für eine aktuellere Schemaversion entwickelt wurde (beispielsweise der Zugriff auf I_1 über SV_2). Vorwärtskompatibilität bezeichnet entsprechend die Fähigkeit, dass Daten, die unter einer neueren Schemaversion erstellt wurden, von einer Anwendung verarbeitet werden können, die für eine ältere Schemaversion konzipiert ist (beispielsweise der Zugriff auf I_2 über SV_1). Zusammenfassend betreffen Rückwärts- und Vorwärtskompatibilität die Möglichkeit, auf Daten über eine andere Schemaversion zuzugreifen als jene, mit der die Daten ursprünglich erzeugt wurden (nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-8).

In der Forschung wird diskutiert, ob Schemaänderungen vollständig verlustfrei und kompatibel umgesetzt werden können. Da Änderungen an Schemata häufig die Menge gültiger Daten verändern, ist vollständige Vorwärts- und Rückwärtskompatibilität theoretisch oft nicht realisierbar. Aus diesem Grund unterstützen viele Systeme lediglich die partielle Schema-Versionierung (nach BRAHMIA, GRANDI, OLIBONI 2024b, S. 2430002-12 f.).

3.1.2 Temporale Schema-Versionierung

Neben nicht-temporalen Konzepten, können Schemata auch entlang zeitlicher Dimensionen versioniert werden. De Castro et al. entwickelten – aufbauend auf einem Vorschlag von John F. Roddick – drei Ansätze der temporalen Schema-Versionierung. Dabei unterscheiden sie zwischen *transaktionszeitbasierter (transaction-time)*, *gültigkeitszeitbasierter (valid-time)* und *bitemporaler (bitemporal-time)* Schema-Versionierung.

Transaktionszeit

Bei der *transaktionszeitbasierten Schema-Versionierung* versieht das DBMS jede Schemaversion mit einem Transaktionsbeginn und einem Transaktionsende als Zeitstempel. Die Kataloginformationen werden dabei analog zu Transaktionszeittabellen verwaltet. Änderungen können ausschließlich an der aktuell gültigen Schemaversion vorgenommen werden. Bereits vergangene Versionen bleiben unveränderlich, während zukünftige Versionen im Rahmen dieses Ansatzes nicht berücksichtigt werden. Dieses Verfahren eignet sich insbesondere für Anwendungsszenarien, in denen Schemaänderungen unmittelbar nach ihrer Durchführung wirksam werden sollen.

Gültigkeitszeit

In der *gültigkeitszeitbasierten Schema-Versionierung* wird jede Version eines Schemas durch einen Beginn sowie ein Ende ihrer zeitlichen Gültigkeit beschrieben. Hierfür verwaltet das DBMS die Kataloginformationen in Form von Gültigkeitszeittabellen. Da sich dieser Ansatz an der Gültigkeitszeit orientiert, lassen sich nicht nur unmittelbar wirksame Änderungen am Schema abbilden, sondern auch rückwirkende sowie zukünftige Anpassungen. Rückwirkende Änderungen entstehen, wenn eine reale Veränderung bereits eingetreten ist und erst nachträglich im Schema nachvollzogen wird. Zukünftige Änderungen hingegen werden bereits vor ihrem tatsächlichen Wirksamwerden definiert und mit einer entsprechenden zukünftigen Gültigkeitsperiode versehen. Dadurch können frühere, aktuelle und geplante Versionen eines Schemas gezielt angepasst werden. Die Versionierung basierend auf Gültigkeitszeit eignet sich insbesondere für Anwendungsbereiche, in denen Schemaänderungen mit rückwirkender oder vorausplanender Wirkung erforderlich sind. Rückwirkende Anpassungen können beispielsweise durch gesetzliche Vorgaben notwendig werden, die strukturelle Änderungen administrativer Datenbestände verlangen – wie

etwa eine Anpassung der Ziffernlänge der Sozialversicherungsnummer. Zukünftige Schemaänderungen bieten dagegen die Möglichkeit, geplante Anpassungen bereits im Vorfeld zu modellieren, zu testen und hinsichtlich ihrer Auswirkungen zu bewerten, bevor sie produktiv eingesetzt werden.

Bitemporale-Zeit

Bei der *bitemporalen Schema-Versionierung* wird jede Version eines Schemas sowohl mit Transaktionszeit- als auch mit Gültigkeitszeitstempeln versehen, sodass die Datenbankkataloge bitemporal organisiert sind. Die Nutzung beider Zeitdimensionen ist insbesondere für Anwendungen relevant, die Schemaänderungen nicht nur zeitnah, sondern auch rückwirkend oder im Voraus ermöglichen müssen und bei denen diese Modifikationen lückenlos nachvollziehbar bleiben sollen. Dadurch wird im Systemkatalog dokumentiert, ob eine Änderung retroaktiv oder proaktiv erfolgt ist. Im Gegensatz dazu erlaubt eine rein gültigkeitszeitbasierte Schema-Versionierung nach der Ausführung keine eindeutige Rekonstruktion, ob eine Schemaänderung rückwirkend oder vorausschauend eingetragen wurde. Durch die zusätzliche Erfassung der Transaktionszeit bleibt hingegen exakt dokumentiert, zu welchem Zeitpunkt die Änderung tatsächlich in das System übernommen wurde.

Im Folgenden werden die Ansätze anhand von Beispielen illustriert. Die Beispiele basieren auf der Arbeit von De Castro et al.

Die DDL-Operationen O_1 bis O_3 werden wie folgt definiert:

- O_1 – Schemaänderung – On-Time-Transaktion – committed am 1/1/1983 – wirksam von 1/1/1983 bis 12/31/1985: Erstellung der Relation *EMPLOYEE* mit den Attributen *NAME*, *ADDRESS* und *CITY*
- O_2 – Schemaänderung – rückwirkende (retroaktive) Transaktion – committed am 1/1/1985 – wirksam von 1/1/1983 bis 12/31/1985: Entfernung des Attributes *ADDRESS* aus der Relation *EMPLOYEE*
- O_3 – Schemaänderung – proaktive Transaktion – committed am 1/1/1987 – wirksam ab 1/1/1988: Hinzufügen des Attributes *PHONE* zur Relation *EMPLOYEE*

Die Abbildungen 8 bis 10 beschreiben den Zustand des DBMS nach Ausführung der DDL-Operationen O_1 bis O_3. Der Einfachheit halber wird eine zeitliche Granularität von einem Tag angenommen. Zudem wird auf eine Angabe der entsprechenden DDL-Statements verzichtet.

Transaktionszeitbasierte Schema-Versionierng

Die Operation O_1 initialisiert das Schema, bestehend aus der Relation *EMPLOYEE* mit den Attributen *NAME*, *ADDRESS* und *CITY*. Gemäß der transaktionszeitbasierten Schema-Versionierung dürfen Anpassungen stets nur die aktuelle Schemaversion betreffen. Schemaänderungen werden daher ausschließlich als On-Time-Transaktionen ausgeführt. Sie werden also mit dem Zeitpunkt ihrer Definition auf das aktuelle Schema wirksam. Damit bleibt eine Schemaversion so lange aktuell, bis eine neue Schemaänderung definiert und damit eine neue Schemaversion erzeugt wird. Der Status im Systemkatalog des DBMS nach O_1 lautet demnach $\mathrm{SV}_1\ (1/1/83 - \infty)$ und bleibt bis zum Zeitpunkt einer neuen Schemaänderung bestehen. Bei den Operationen O_2 und O_3 handelt es sich um retro- bzw. proaktive Schemaänderungen. Da die transaktionszeitbasierte Versionierung diese Art der Anpassung nicht unterstützt, werden auch diese Schemaänderungen erst zum Zeitpunkt ihrer Definition wirksam (vgl. Abb. 8). O_2 erzeugt die Schemaversion SV_2 und O_3 die Schemaversion SV_3.

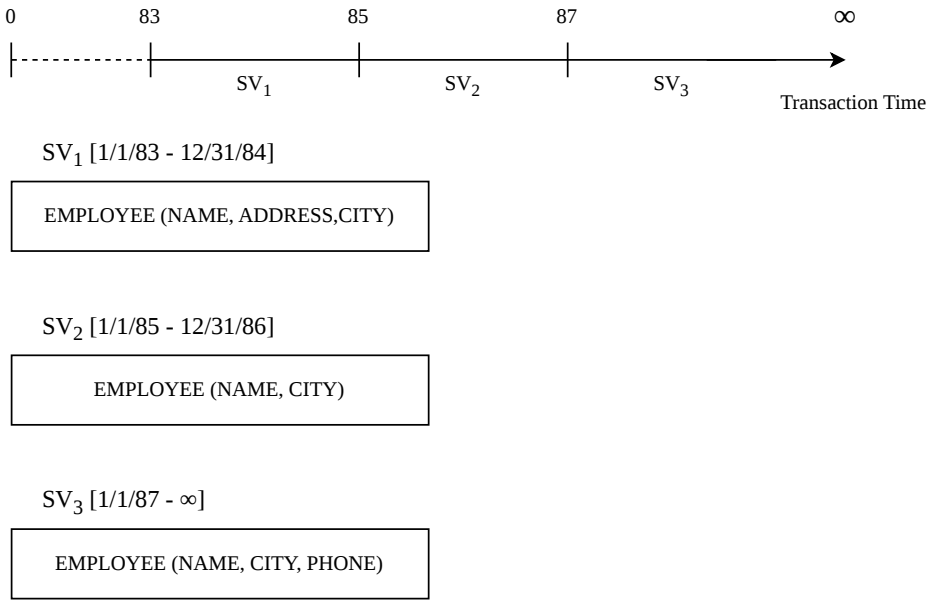


Abbildung 8: Transaktionszeitbasierte Schema-Versionierung

Gültigkeitszeitbasierte Schema-Versionierung

Im Gegensatz zur Versionierung nach der Transatkionszeit, kann bei der gültigkeitszeitbasierten Schema-Versierung der Gültigkeitszeitraum der Schemaänderung explizit definiert werden. Die Anpassungen betreffen dann jenes Schema, das für diesen spezifischen Gültigkeitszeitraum definiert ist. Dadurch sind Modifikationen an vergangenen oder zukünftigen Schemaversionen möglich. Da die retroaktive Schemaänderungen O_2 für den gleichen Gültigkeitszeitraum definiert ist wie die Schemaversion SV_1, ersetzt die aus O_2 resultierende Version SV_2 die alte Version SV_1 vollständig. SV_1 wird in diesem Zuge gelöscht und ist in der Datenbank somit nicht mehr existent. Die proaktive Schemaänderung O_3 ist wiederum für einen Gültigkeitszeitraum in der Zukunft definiert. O_3 wird somit erst am 01.01.1988 wirksam, wodurch SV_3 entsteht. In der Arbeit von De Castro et al. wird allerdings nicht näher erläutert, welche Version im Zeitraum von 1986-1988 greift.

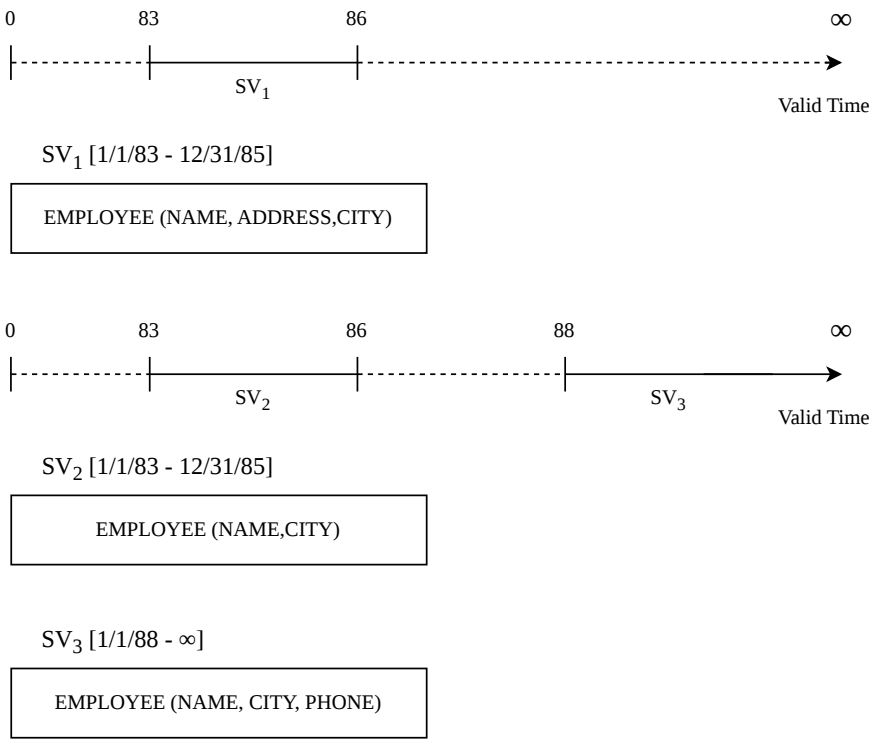


Abbildung 9: Gültigkeitszeitbasierte Schema-Versionierung

Bitemporale Schema-Versionierung

Die Versionierung von Schemata entlang der Transaktions- und der Gültigkeitszeit ermöglicht es, historische Stände der Datenbank vollständig zu bewahren. Ein Rückgang auf ältere Schemaversionen ist somit jederzeit möglich. Schemata die für denselben Gültigkeitszeitraum definiert sind, müssen nicht mehr gelöscht werden, da die Transaktionszeit eindeutig belegt, welche Schemaversion zu welchem Zeitpunkt systemaktuell war. Die aus O_2 resultierende Version SV_2 löst die Version SV_1 hinsichtlich ihrer Aktualität zwar ab; SV_1 wird nach diesem Ansatz jedoch nicht gelöscht, sondern archiviert und bleibt weiterhin abrufbar, sofern ein Rückgang von SV_2 auf SV_1 erforderlich ist. Sowohl die intensionalen (Schema) als auch die extensionalen Daten (Instanz) bleiben dabei vollständig erhalten.

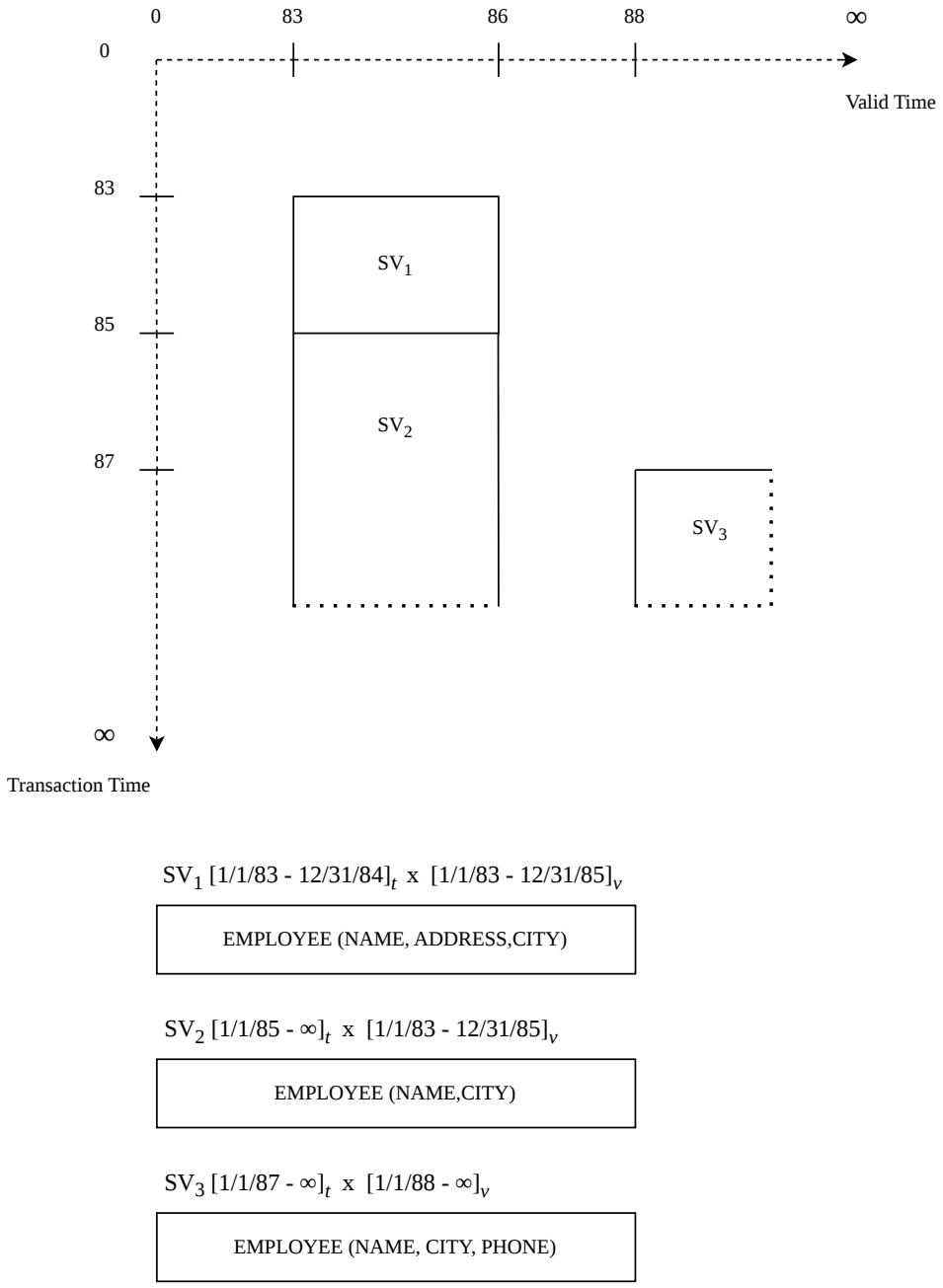


Abbildung 10: Bitemporale Schema-Versionierung

3.1.3 Speicherkonzept

Hinsichtlich der Koordination zwischen Schemaversion und Instanz unterscheiden De Castro et al. zwei grundlegende Speicherstrategien: die *Single-Pool-* und die *Multi-Pool-*

Speicherung. Der Begriff *Data-Pool* definiert in diesem Kontext einen logischen Speicherort für extensionale Daten. Beide Ansätze sind primäre auf der logischen Ebene zu betrachten, da die Autoren Details der physischen Speicherung bewusst abstrahieren.

Bei der *Multi-Pool-Speicherung* werden die Daten jeder Version einer Relation in einem separaten, isolierten Speicherbereich verwaltet. Die Datenbestände der unterschiedlichen Versionen werden somit unabhängig voneinander persistiert. Bezogen auf das Beispiel aus Kapitel 2.4 entspricht das Ablegen der Instanzen I_1 und I_2 in jeweils eigenen Tabellen einer Speicherung nach dem Multi-Pool-Verfahren (vgl. Abb. 11).

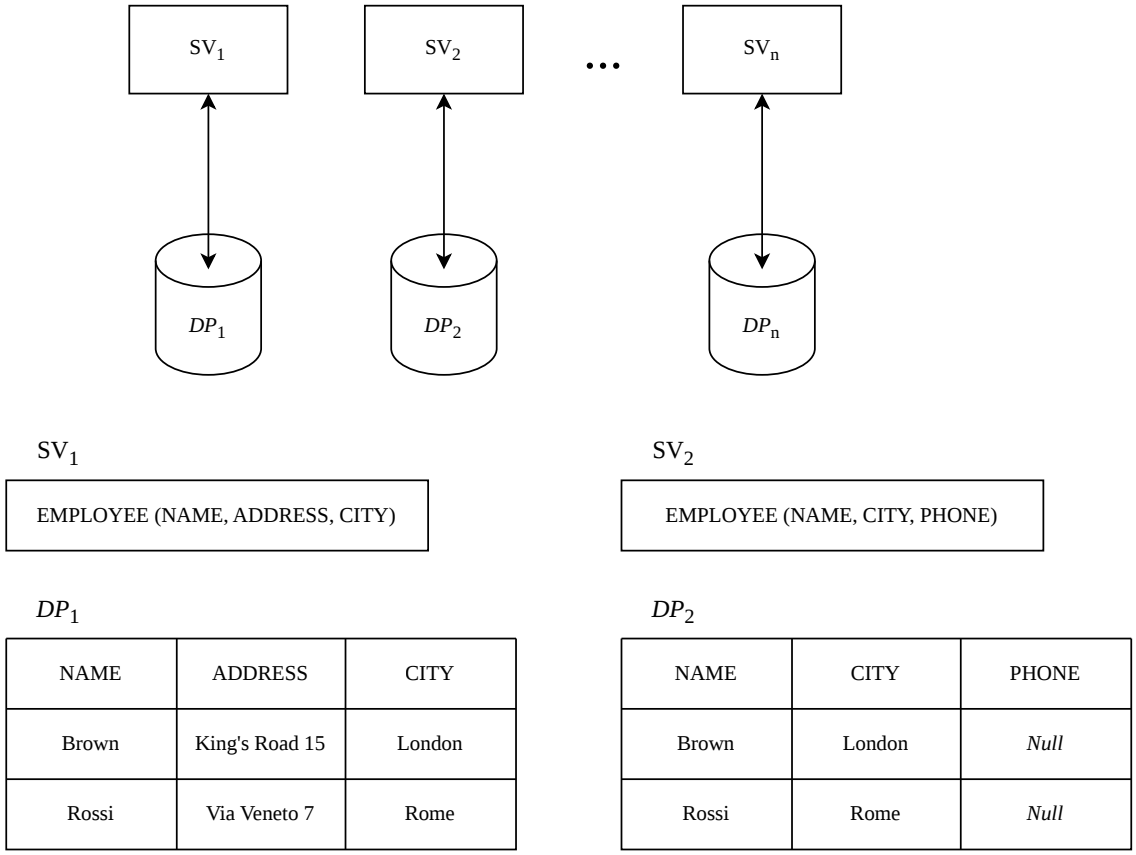


Abbildung 11: Multi-Pool-Speicherung

Die *Single-Pool-Speicherung* verfolgt hingegen das Ziel, sämtliche Daten einer Relation zentral in einer gemeinsamen Struktur zu verwalten. Dadurch lassen sich Rendundanzen vermeiden - insbesondere dann, wenn mehrere Schemaversionen identische Datenwerte teilen. Für die Umsetzung des Single-Pool-Ansatzes wird ein übergeordnetes Schema

erzeugt, welches die Vereinigung sämtlicher Attribute aller existierenden Schemaversionen der Relation darstellt. Dieses sogenannte *Completed Schema* dient als gemeinsame logische Basis für die Datenspeicherung. Die einzelnen Versionen des Schemas werden folglich nicht mehr als eigenständige Tabellen geführt, sondern als Views auf dieser gemeinsamen Datenbasis definiert. Jede View projiziert dabei exakt jene Attribute, welche für die jeweilige Schemaversion relevant sind. Abbildung 12 illustriert eine solche Single-Pool-Implementierung für das Beispiel aus Kapitel 2.4, bei der die Instanzen von SV_1 und SV_2 in einer einzigen, konsolidierten Tabelle gehalten werden.

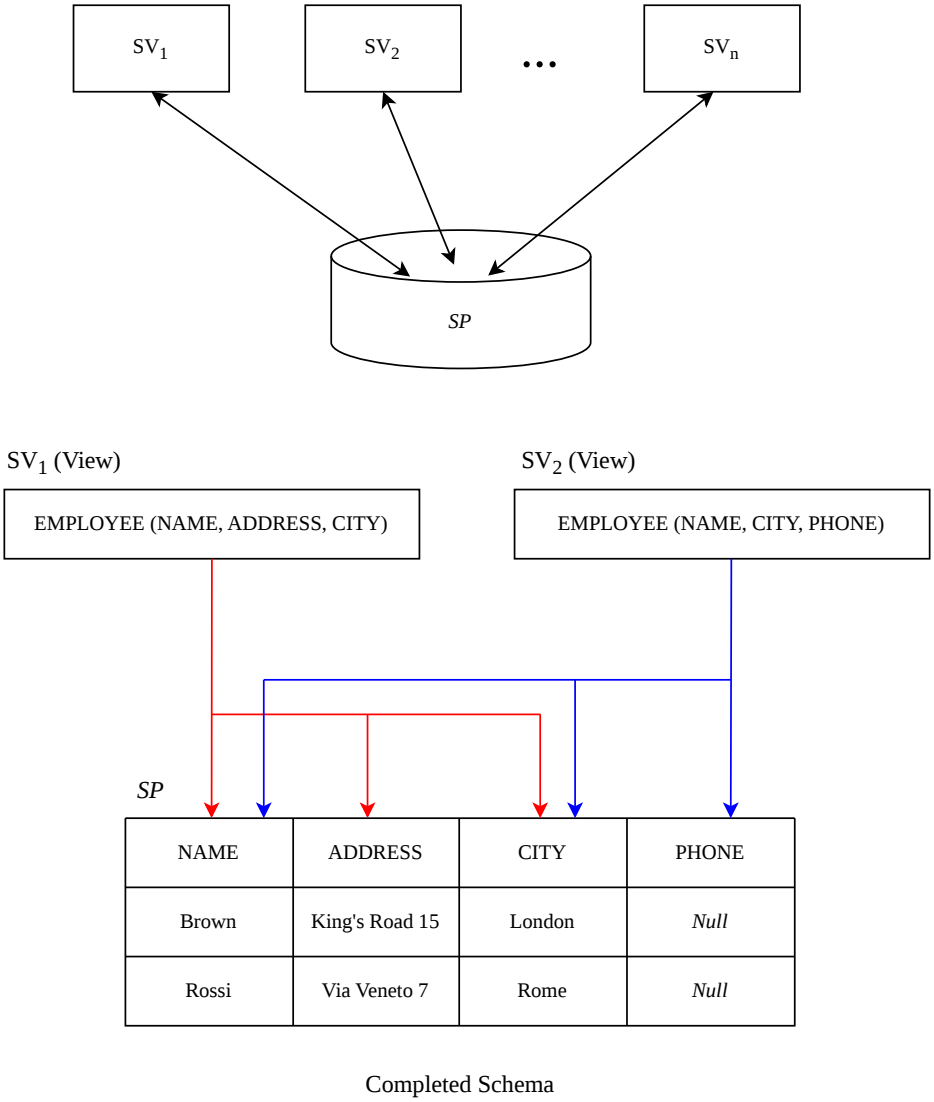


Abbildung 12: Single-Pool-Speicherung

Ein wesentlicher Unterschied zwischen beiden Ansätzen zeigt sich im Aufwand zur Gewährleistung von Vorwärts- und Rückwärtskompatibilität. Existieren insgesamt n Schemaversionen, so erfordert der Single-Pool-Ansatz lediglich n Views, um die Datenbestände zwischen den verschiedenen Versionen interoperabel zugänglich zu machen und somit sowohl die Vorwärts- als auch die Rückwärtskompatibilität vollständig zu unterstützen. Beim Multi-Pool-Ansatz hingegen skaliert der Aufwand quadratisch: Da jede Version potenziell mit jeder anderen Version interagieren muss, können im Worst-Case insgesamt

$n \cdot (n-1)$ Views notwendig werden.

Auch hinsichtlich der Modifizierbarkeit der Daten ergeben sich signifikante Unterschiede. Zur Unterstützung einer partiellen Schema-Versionierung muss im Single-Pool-Modell das jeweils aktuelle Schema als aktualisierbare View auf dem übergeordneten Completed Schema definiert sein. Soll dagegen eine vollständige Schema-Versionierung realisiert werden, setzt dies voraus, dass sämtliche im System existierenden Views aktualisierbar sind, was eine wesentlich komplexere Anforderung darstellt.

Weiterführende Vor- und Nachteile beider Speicherstrategien wurden von De Castro et al. und Brahmia et al. beleuchtet. Auf diese tiefergehenden Analysen wird an dieser Stelle verwiesen; eine detaillierte Ausarbeitung würde jedoch den Rahmen der vorliegenden Arbeit überschreiten.

3.1.4 Interaktion zwischen Instanz und Schema

In den bisherigen Betrachtungen wurden ausschließlich Snapshot-Daten herangezogen, da diese hinreichend waren, um grundlegenden Mechanismen der Schema-Versionierung zu erläutern. Zusätzliche Herausforderungen entstehen jedoch, wenn Schema-Versionierung in Datenbanken eingesetzt wird, die temporale Daten verwalten. In diesem Fall muss das Zusammenspiel zwischen der Versionierung der Daten selbst (extensionale Versionierung) und der Versionierung des Schemas (intensionale Versionierung) berücksichtigt werden. Aus dieser wechselseitigen Abhängigkeit resultiert eine weitere architektonische Gestaltungsoption für das Datenmanagement. Diese wird relevant, wenn sowohl die Instanz als auch das Schema entlang derselben Zeitdimension versioniert werden. Die Fachliteratur differenziert in diesem Zusammenang zwischen zwei fundamentalen Verwaltungsansätzen: *Synchrones Management* und *Asynchrones Management*

Synchrones Management

Temporale Daten werden ausschließlich über jene Schemaversion verarbeitet, deren zeitlicher Gültigkeitsbereich mit dem der Daten übereinstimmt. Das betrifft sowohl das Speichern als auch das Abrufen und Aktualisieren der Daten.

Asynchrones Management

Temporale Daten können unabhängig von ihrer zeitlichen Einordnung über beliebige Schemaversionen abgefragt oder verändert werden. Die zeitliche Gültigkeit von Daten und Schema muss hierbei also nicht übereinstimmen. Dieses Modell ist vor allem notwendig, um Rückwärts- und Vorwärtskompatibilität zu ermöglichen sowie partielle oder vollständige Schema-Versionierung zu unterstützen, wenn sowohl Daten als auch Schemata entlang derselben zeitlichen Dimension versioniert werden.

In diesem Kontext erwähnen die Autoren eine wichtige Einschränkung. Werden Schema und Daten über die Transaktionszeit verwaltet, ist nur synchrones Management möglich, weil diese Zeitdimension den tatsächlichen historischen Zustand der Datenbank beschreibt. Wenn eine Datenbank auf einen bestimmten Zeitpunkt zurückgesetzt wird, muss sowohl das Schema als auch die gespeicherten Daten genau zu diesem Zeitpunkt passen. Würde man Schema- und Datenversionen asynchron verwalten, könnten unterschiedliche zeitliche Zustände miteinander kombiniert werden — etwa ein Schema aus dem Jahr 1995 mit Daten aus dem Jahr 1996. Dadurch entstünde ein inkonsistenter Datenbankzustand, der historisch nie tatsächlich existiert hat. Genau das widerspricht jedoch der Bedeutung der Transaktionszeit, deren Zweck darin besteht, vergangene Zustände der Datenbank korrekt und vollständig rekonstruieren zu können. Deshalb müssen Schema und Daten entlang der Transaktionszeit immer gemeinsam und zeitlich abgestimmt versioniert sowie abgefragt werden. Nur so bleibt gewährleistet, dass jeder rekonstruierte Zustand konsistent ist. Für die Gültigkeitszeit gilt diese Einschränkung dagegen nicht, da sie nicht den historischen Zustand der Datenbank selbst beschreibt, sondern die fachliche Gültigkeit von Informationen in der realen Welt.

Das nachfolgende Beispiel soll die Funktionweisen beider Verwaltungsansätze verdeutlichen. Der Ausgangspunkt ist die Relation *EMPLOYEE* mit den Attributen *NAME* und *SALARY*. Sowohl Schema als auch Daten werden über die Gültigkeitszeit-Dimension versioniert. Die zwei Tupel zeigen die Gehaltshistorie des Mitarbeiters *Brown* (vgl. Abb. 13).

SV_1 [1/1/90 - ∞]

EMPLOYEE (NAME, SALARY | T)

I_1

NAME	SALARY	T
Brown	1000$	{1/1/92...12/31/94}
Brown	1500$	{1/1/95...∞}

Abbildung 13: Gehaltshistorie des Mitarbeiter *Brown*

Anschließend wird eine Schemaänderung angenommen, bei welcher der Relation EMPLOYEE das zusätzliche Attribut DEPT hinzugefügt wird. Diese Änderung soll ab dem 1. Januar 1994 gültig sein. Darüber hinaus wird angenommen, dass dieselbe Transaktion mithilfe der neuen Schemaversion die Information speichert, dass der Mitarbeiter Brown schon immer in der Sales-Abteilung gearbeitet hat.

Abbildung 14 veranschaulicht den Zustand des DBMS nach der Schemaänderung unter den Bedingungen des synchronen Managements. Es zeigt sich, dass die Abteilungsinformation *Sales* für den Zeitraum 1992–1993 nicht gespeichert werden konnte, da die entsprechenden Daten von *Brown* in diesem Zeitraum noch über die Schemaversion SV_1 verwaltet werden, die das Attribut *DEPT* nicht enthält. Darüber hinaus müssten spezielle Anwendungen entwickelt werden, die beide Schemaversionen berücksichtigen, um die vollständige Historie der älteren Daten – also die Daten ohne das Attribut *DEPT* – von 1990 bis heute rekonstruieren zu können. Der Grund dafür liegt darin, dass die Historie der Daten im synchronen Management analog zu den Gültigkeitsintervallen der Schemaversionen partitioniert wird.

SV_1 [1/1/90 - 12/31/93]

EMPLOYEE (NAME, SALARY | T)

I_1

NAME	SALARY	T
Brown	1000$	{1/1/92...12/31/93}

SV_2 [1/1/94 - ∞]

EMPLOYEE (NAME, SALARY, DEPT | T)

I_2

NAME	SALARY	DEPT	T
Brown	1000$	Sales	{1/1/94...12/31/94}
Brown	1500$	Sales	{1/1/95...∞}

Abbildung 14: Synchrones Management

Abbildung 15 zeigt den Zustand des DBMS unter den Bedingungen des asynchronen Managements. In diesem Fall bleibt die vollständige Historie der Mitarbeiterdaten in jeder Schemaversion erhalten. Bestehende Anwendungen können weiterhin mit den älteren Daten arbeiten und liefern dabei dieselben Ergebnisse wie vor der Schemaänderung. Gleichzeitig können neue Anwendungen entwickelt werden, die auf den erweiterten Datenbestand zugreifen und zusätzlich die Informationen des Attributs *DEPT* nutzen.

SV_1 [1/1/90 - 12/31/93]

EMPLOYEE (NAME, SALARY | T)

I_1

NAME	SALARY	T
Brown	1000$	{1/1/92...12/31/94}
Brown	1500$	{1/1/95...∞}

SV_2 [1/1/94 - ∞]

EMPLOYEE (NAME, SALARY, DEPT | T)

I_2

NAME	SALARY	DEPT	T
Brown	1000$	Sales	{1/1/92...12/31/94}
Brown	1500$	Sales	{1/1/95...∞}

Abbildung 15: Asynchrones Management

An dieser Stelle sei noch erwähnt, dass die konkreten Auswirkungen von Schema- und Datenänderungen im Rahmen des asynchronen Managements zusätzlich davon abhängen, ob die Daten mithilfe des Single-Pool- oder des Multi-Pool-Ansatzes verwaltet werden. Zudem wird vollständige Schema-Versionierung bei einer Versionierung entlang der Transaktionszeit aus den genannten Konsistenzgründen ausgeschlossen.

Abschließend sei darauf hingewiesen, dass viele wissenschaftliche Lösungsansätze auf einer Kombination der zuvor vorgestellten Dimensionen und Konzepte beruhen. Aufgrund verschiedener technischer und konzeptioneller Restriktionen sind jedoch nicht alle Kombinationen miteinander kompatibel. Werden Daten und Schemata entlang derselben Zeitdimension versioniert, stellt die Gültigkeitszeit in Verbindung mit asynchronem Management die einzige Kombination dar, die partielle oder vollständige Schema-Versionierung ermöglicht.

Ein Beispiel für einen Lösungsansatz, der eine vollständige Schema-Versionierung unterstützt, ist der Prototyp VERSIOS. Dieser basiert auf einer Versionierung entlang der Gültigkeitszeit, verwendet die Multi-Pool-Speicherung und verwaltet die Interaktion zwischen Schema- und Instanzversionen mittels asynchronem Management.

3.2 DBMS Support und Tooling

Auch wenn die Schema-Versionierung theoretisch die einzige Methode darstellt, mit der sich Änderungen an einer sich entwickelnden Datenbankstruktur konsistent nachverfolgen und frühere Zustände rekonstruieren lassen, wird dieses Konzept von heutigen relationalen DBMS nicht nativ unterstützt. Auch der SQL-Standard definiert bislang kein Konzept für Schema-Versionierung.

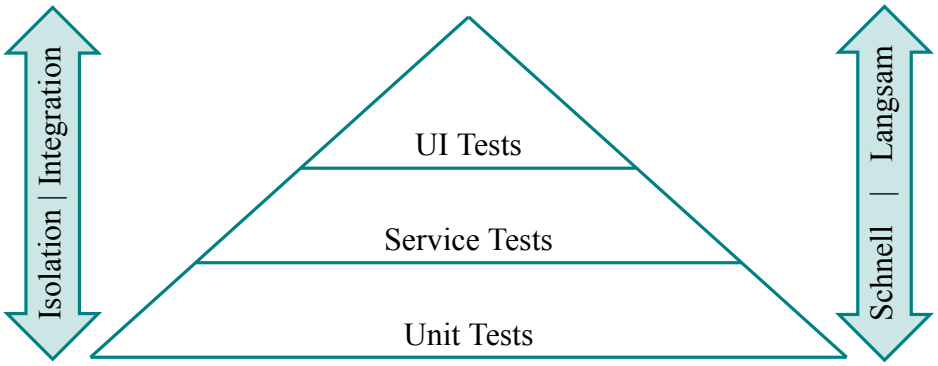
Viele kommerzielle Datenbanksysteme wie Microsoft SQL Server, IBM DB2, Oracle oder MySQL stellen jedoch über integrierte Funktionen oder die Unterstützung externer Werkzeuge eine Versionsverwaltung für die Entwicklung von Schemata zur Verfügung. Dadurch können Entwickler parallel an einem noch nicht produktiven Schema arbeiten und verschiedene Varianten bzw. aufeinanderfolgende Versionen erzeugen. Diese Mechanismen unterstützen beispielsweise das Zusammenführen von Änderungen verschiedener Entwickler, das gezielte Zurücknehmen fehlerhafter Anpassungen, die Nachvollziehbarkeit von Unterschieden zwischen Entwicklungsständen oder Schema-Varianten sowie die Erzeugung von Skripten zur Überführung eines Schemas in eine andere Version. Eine solche Überführung kann auch bestehende Daten betreffen und stellt damit eine partielle und oft verlustbehaftete Unterstützung der Schema-Evolution vor der Produktivsetzung dar. Tools wie Flyway oder Liquibase unterstützen diese Funktionalität. Obwohl diese Tools häufig der Schema-Versionierung zugeordnet werden, handelt es sich dabei nicht um Schema-Versionierung im Sinne der Endnutzer: In produktiven Umgebungen können bestehende Daten nicht gleichzeitig über verschiedene Schemaversionen hinweg geteilt oder unterschiedlich angezeigt und bearbeitet werden.

Eine Lösung die dem Konzept der Schema-Versionierung vergleichsweise nahekommt, ist der *Oracle Workspace Manager*.

Der *Oracle Workspace Manager* (kurz *OWM*) stellt eine datenbankinterne Versionierungsmechanik bereit, die auf Zeilenebene mehrere konsistente Datenzustände innerhalb sogenannter *Workspaces* ermöglicht. Im Gegensatz zu klassischen Migrationsansätzen erlaubt das System damit eine parallele Existenz unterschiedlicher Datenversionen inner-

halb derselben Datenbankinstanz. Dadurch ist OWM der Schema-Versionierung konzeptionell sehr nahe, da nicht nur eine zeitliche Evolution, sondern auch gleichzeitige isolierte Zustände unterstützt werden. Dennoch handelt es sich nicht um eine vollständige Schema-Versionierung, da insbesondere die strukturelle Unabhängigkeit von Instanzen und Schemata nicht gegeben ist und Versionierung primär auf Daten- statt auf Strukturebene erfolgt.

Abbildung 16: Testpyramide



Quelle: Eigendarstellung angelehnt an [1, S. 311–317]

3.3 Ausblick

Die standardisierte SQL-DDL, die für die Definition und Modifikation relationaler Datenbankschemata eingesetzt wird, weist lediglich eingeschränkte Möglichkeiten zur Unterstützung von Schemaänderungen auf und bietet keine native Funktionalität für Schema-Evolution oder Schema-Versionierung. Aus diesem Grund erscheint eine Erweiterung der SQL-Spezifikation sinnvoll, die explizit Konzepte der Schema-Evolution und Versionierung auf Benutzerebene unterstützt. Eine solche Weiterentwicklung könnte dazu beitragen, Änderungen am Datenbankschema nicht nur technisch, sondern auch konzeptionell transparenter und benutzerfreundlicher abzubilden. Im Hinblick auf zukünftige Forschungsarbeiten eröffnet sich hierbei ein breites Spektrum an Fragestellungen. Insbesondere die Entwicklung standardisierter Sprachkonstrukte für Schema-Versionierung, die Integration von Migrationsmechanismen direkt in die Datenbanksprache sowie die konsistente Unterstützung historisierter Datenabfragen über mehrere Schemazustände hinweg stellen vielversprechende Forschungsrichtungen dar. Darüber hinaus wäre zu untersuchen, wie sich solche Erweiterungen in bestehende Datenbankmanagementsysteme integrieren lassen, ohne deren Komplexität oder Performance wesentlich zu beeinträchtigen.

Literaturverzeichnis

[1] M. Cohn, *Succeeding with agile: software development using Scrum*, English. Upper Saddle River, N.J.: Addison-Wesley, 2010, OCLC: 489135509, ISBN: 9780321660510 9786612430565 9781282430563 9780321660565. Adresse: http://search.ebscohost.com/login.aspx?direct=true&scope=site&db=nlebk&db=nlabk&AN=1599331.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die angemeldete Prüfungsleistung in allen Teilen eigenständig ohne Hilfe von Dritten anfertigen und keine anderen als die in der Prüfungsleistung angegebenen Quellen und zugelassenen Hilfsmittel verwenden werde. Sämtliche wörtlichen und sinngemäßen Übernahmen inklusive KI-generierter Inhalte werde ich kenntlich machen.

Diese Prüfungsleistung hat zum Zeitpunkt der Abgabe weder in gleicher noch in ähnlicher Form, auch nicht auszugsweise, bereits einer Prüfungsbehörde zur Prüfung vorgelegen; hiervon ausgenommen sind Prüfungsleistungen, für die in der Modulbeschreibung ausdrücklich andere Regelungen festgelegt sind.

Mir ist bekannt, dass die Zuwiderhandlung gegen den Inhalt dieser Erklärung einen Täuschungsversuch darstellt, der das Nichtbestehen der Prüfung zur Folge hat und daneben strafrechtlich gem. § 156 StGB verfolgt werden kann. Darüber hinaus ist mir bekannt, dass ich bei schwerwiegender Täuschung exmatrikuliert und mit einer Geldbuße bis zu 50.000 EUR nach der für mich gültigen Rahmenprüfungsordnung belegt werden kann.

Ich erkläre mich damit einverstanden, dass diese Prüfungsleistung zwecks Plagiatsprüfung auf die Server externer Anbieter hochgeladen werden darf. Die Plagiatsprüfung stellt keine Zurverfügungstellung für die Öffentlichkeit dar.

Köln, 24.05.2026

(Ort, Datum)

(Eigenhändige Unterschrift)